

Run and test the Telegram Physical Goods Shop bot

Step by step, with **cash on delivery / pickup only** (MIA left disabled). Leave MIA_RECIPIENT, MIA_PHONE and MIA_IBAN empty and the bot does not offer MIA at all, so no MIA code path runs.

1. Create a test bot

- 1 In Telegram open **@BotFather**, send /newbot and copy the token.
- 2 Get your numeric Telegram ID from **@myidbot**.
- 3 Use a separate test bot, not your real shop bot.

2. Configure

```
git clone https://github.com/aventurine-tech/Telegram-shop
cd Telegram-shop
git checkout claude/great-clarke-cep0m5
cp .env.example .env
```

Edit **.env** and set:

```
TOKEN=<token from BotFather>
OWNER_ID=<your Telegram ID>
BOT_LOCALE=ro          # or en / ru
POSTGRES_PASSWORD=<any strong password>
REDIS_PASSWORD=<any password>
ADMIN_PASSWORD=<anything except "admin">
SECRET_KEY=<python -c "import secrets; print(secrets.token_hex(32))">

MIA_RECIPIENT=
MIA_PHONE=
MIA_IBAN=
COD_ENABLED=1
DELIVERY_ENABLED=1
PICKUP_ENABLED=1
PICKUP_ADDRESS=Str. Test 1, Chisinau
```

- **ADMIN_PASSWORD** and **SECRET_KEY** must be changed. In Docker the web panel is reachable, so the bot refuses to start while either keeps its default.
- Keep REDIS_PASSWORD non-empty when using the bundled Redis.

3. Start it

```
docker compose up -d --build
docker compose logs -f bot
```

Wait for the log to show the bot started. The container applies the database migrations itself. If it exits, read the log: a startup error names the problem. (Without Docker you need a running Postgres, then: pip install -r requirements.txt, alembic upgrade head, python run.py.)

4. Set up the shop (as owner)

- 1 Send /start to your bot from your own account. You get the owner role.
- 2 Open **Admin panel**, then **Categories**, and create a category.
- 3 Under **Goods** (products) add a product: name, description, price, category and **stock** (for example 5).

- 4 Optional: open `http://localhost:9090/admin` and log in with `ADMIN_USERNAME / ADMIN_PASSWORD` to see the same data.

5. Place a test order (as a customer)

Use a **second Telegram account**.

- 1 `/start`, then **Shop**, then the category and the product. The card shows the price and the stock.
- 2 Add to cart (or "Order now"), open the cart, change the quantity, then check out.
- 3 Choose **Delivery** or **Pickup**, then enter name, phone, address (delivery only) and an optional comment.
- 4 The payment step should offer **only cash on delivery/pickup**. Confirm the order.

6. Check what should happen

- **Stock:** the product's stock drops by the quantity ordered.
- **Staff alert:** your owner account receives a "new order" message with an open-order button.
- **My orders:** the customer sees the order under Profile, then My orders.
- **Walk the order through:** Admin panel, then Orders. Press Confirm, then Mark shipped, then Complete. The customer gets a message at each step.
- **Cash:** completing the order marks the cash as collected.
- **Cancel:** place a second order and cancel it from the admin side. The stock comes back.
- **Self-cancel:** place a third order and cancel it as the customer from My orders.
- **Out of stock:** order the last units; the card shows out of stock with a notify-me button. Restocking from the admin menu messages that user.

7. Optional extras

- **Referral:** set `REFERRAL_PERCENT=10`, restart the bot, and use another account's referral link. Completing that account's order credits the referrer's balance.
- **Balance:** Admin, Users, open a customer and add balance. The customer can then use it at checkout.
- **Language:** change `BOT_LOCALE`, then `docker compose restart bot`.

Handy commands

<code>docker compose logs -f bot</code>	<code># live log</code>
<code>docker compose restart bot</code>	<code># after editing .env</code>
<code>docker compose down</code>	<code># stop</code>
<code>docker compose down -v</code>	<code># stop and wipe the database</code>

Button labels may differ slightly from the names used here, because they come from the translation files. If a step fails, check the bot log first.